Modelli di processo per lo sviluppo del software
I processi agili



Corso di Ingegneria del Software
Laurea in Informatica · Università di Bologna

Obiettivi della lezione

- Spiegare il ruolo di collaborazione, feedback e adattamento.
- Distinguere valori agili, pratiche XP, Scrum e Lean Startup.
- Scrivere una user story con criteri di accettazione verificabili.
- Collegare TDD, refactoring e integrazione continua.
- Valutare il supporto dell’IA generativa senza delegare le decisioni.

Sviluppare software è un’attività sociale

- Lo sviluppo combina difficoltà tecniche e organizzative.
- Comprendere i bisogni richiede conversazioni e negoziazione.
- Il team coordina competenze, dipendenze e decisioni.
- Un prodotto corretto deve anche risolvere un problema utile.

Le competenze tecniche e la collaborazione si rafforzano reciprocamente.

Lavorare per progetti e per prodotti

Prospettiva	Progetto	Prodotto
Orizzonte	Iniziativa temporanea con un obiettivo.	Evoluzione nel tempo e ciclo di vita.
Pianificazione	Vincoli di ambito, tempi e risorse.	Obiettivi di prodotto e lavoro ordinato per valore.
Successo	Raggiungimento degli obiettivi del progetto.	Valore per utenti e organizzazione nel tempo.

Un progetto può essere agile e avere più rilasci. Le due prospettive possono coesistere.

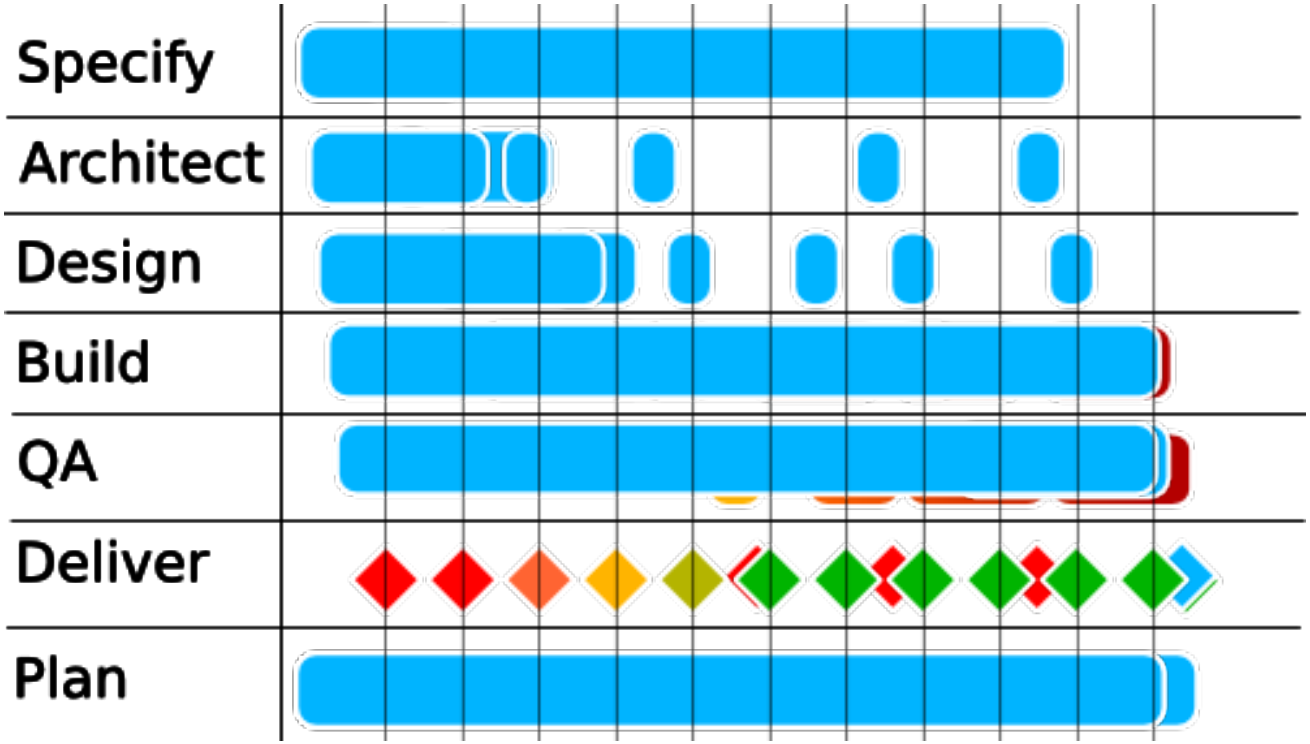
Pianificare in presenza di cambiamenti

- All’inizio conosciamo solo una parte del problema.
- Il feedback può modificare requisiti e priorità.
- Anche vincoli, tecnologie e condizioni esterne possono cambiare.
- Pianificare significa formulare una previsione e aggiornarla con nuove informazioni.

L’agilità richiede pianificazione frequente e verificabile.

Quando partecipa l’utente?

Dove raccogliamo e usiamo il feedback?



Product Backlog e Sprint Backlog

Artefatto Scrum	Contenuto	Responsabilità
Product Backlog	Lavoro necessario per migliorare il prodotto: funzionalità, difetti, attività tecniche.	Il Product Owner risponde della sua gestione e del suo ordinamento.
Sprint Backlog	Sprint Goal, elementi selezionati e piano per realizzarli.	I Developers creano e aggiornano il piano durante lo Sprint.

La selezione del lavoro è una previsione. Il backlog non contiene soltanto user stories.

I quattro valori del Manifesto Agile

Attribuiamo più valore a…	**…pur riconoscendo il valore di**
Individui e interazioni	Processi e strumenti
Software funzionante	Documentazione esaustiva
Collaborazione col cliente	Negoziazione dei contratti
Rispondere al cambiamento	Seguire un piano

La preferenza tra i valori non elimina strumenti, documentazione, contratti o piani.

I dodici principi: quattro temi

Tema	Principi collegati
Valore e adattamento	Consegne anticipate e frequenti, software funzionante, cambiamenti benvenuti.
Collaborazione e autonomia	Lavoro quotidiano con il business, persone motivate, conversazioni, team auto-organizzati.
Qualità e sostenibilità	Ritmo sostenibile, eccellenza tecnica e buon design.
Semplicità e miglioramento	Massimizzare il lavoro non necessario evitato; riflettere e adattare il modo di lavorare.

Feedback su tempi diversi

Fonte del feedback	Domanda	Scala tipica
Test automatici	Il comportamento atteso è preservato?	Secondi o minuti
Coppia e integrazione	Il codice è comprensibile e si integra?	Durante il lavoro e ogni giorno
Cliente e utenti	L'incremento risolve il problema?	Durante l'iterazione e ai rilasci
Riflessione del team	Il nostro modo di lavorare è efficace?	A intervalli regolari

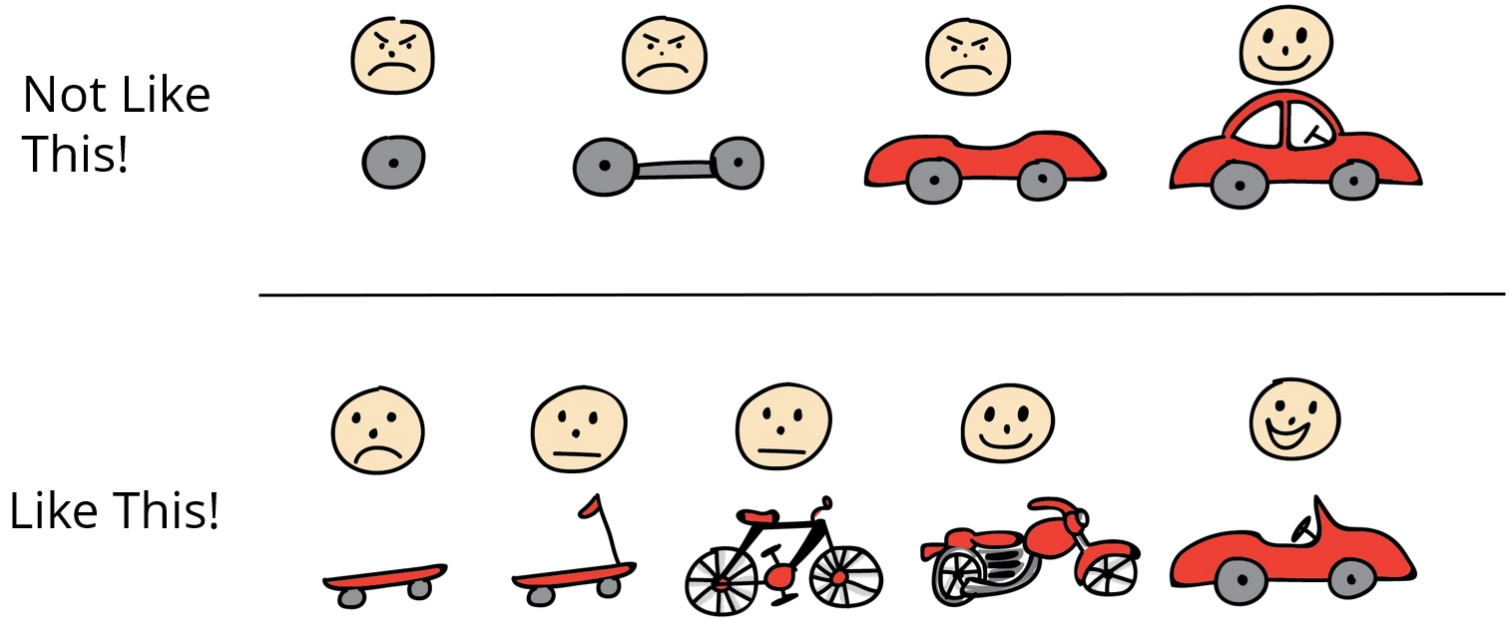
Tempi indicativi: conta quanto rapidamente il team può usare il riscontro.

Minimum Viable Product: apprendere con poco sforzo

- Un MVP mette alla prova un'ipotesi sul prodotto e sui suoi utenti.
- Il risultato serve a decidere se proseguire, cambiare direzione o fermarsi.
- "Minimo" riguarda lo sforzo necessario per ottenere un riscontro utile.
- "Viable" richiede un'esperienza sufficiente a rendere significativo il riscontro.

È un concetto del Lean Startup. Il solo numero di funzionalità non definisce un MVP.

MVP e incrementi utilizzabili



La figura mostra utilità a ogni passo. Per un MVP servono anche ipotesi e osservazioni.

Dropbox: una dimostrazione prima del prodotto completo



Una demo può rendere comprensibile l'esperienza proposta prima di costruire tutte le funzionalità.

Domande: quale ipotesi si verifica? Quale risposta degli utenti sarebbe informativa?

Video dimostrativi:

https://www.youtube.com/watch?v=qxFLfY7_Gqw
https://www.youtube.com/watch?v=xy9nSnalvPc

Approcci collegati all’agilità

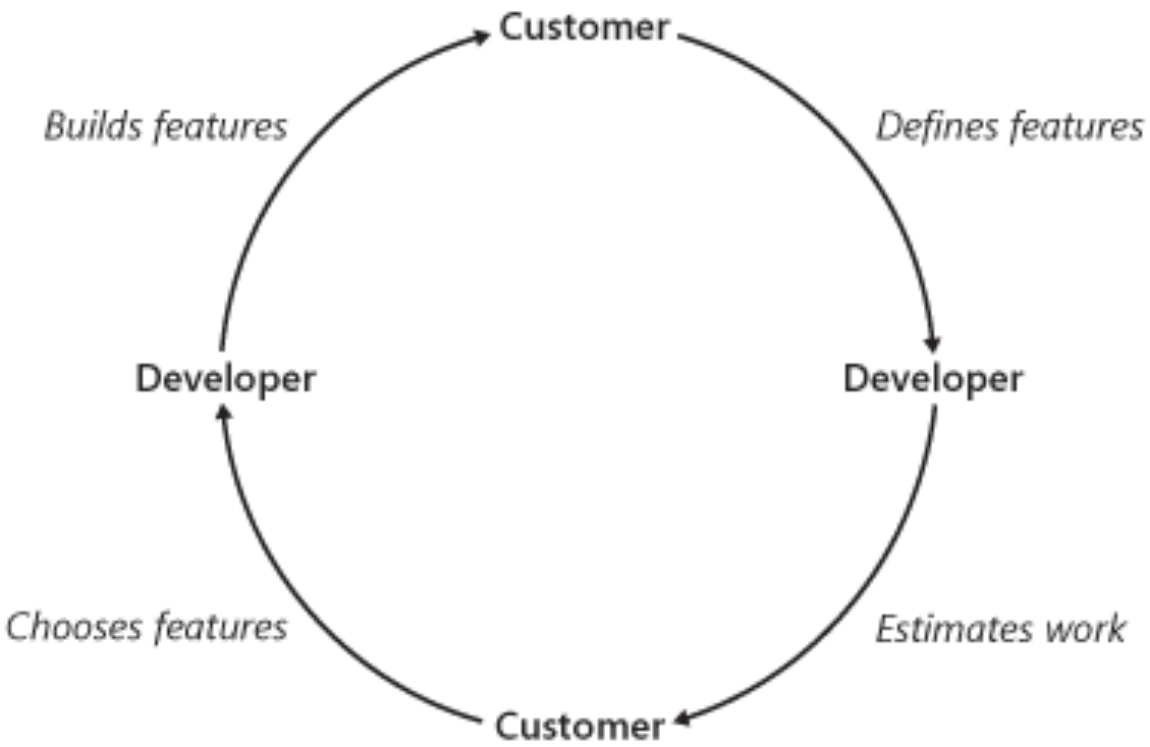
Categoria	Esempi	Focus
Metodi e framework di sviluppo	XP, Scrum, Crystal, FDD, ASD, DSDM	Organizzazione dello sviluppo e pratiche del team
Gestione del flusso	Kanban, Lean software development	Flusso di lavoro, capacità e sprechi
Framework per più team	SAFe, LeSS	Coordinamento su scala organizzativa
Sviluppo e operazioni	DevOps	Collaborazione, automazione e gestione del servizio

Sono approcci con scopi diversi. In questa lezione approfondiamo XP.

I valori di Extreme Programming

Valore	Implicazione nel lavoro
Comunicazione	Rendere visibili conoscenze, problemi e decisioni.
Semplicità	Costruire ciò che serve oggi con un design comprensibile.
Feedback	Usare riscontri dal codice, dai test e dalle persone.
Coraggio	Affrontare problemi, modificare il codice, dichiarare difficoltà.
Rispetto	Riconoscere il contributo e i bisogni delle altre persone.

La collaborazione tra cliente e sviluppatori



Il cliente porta priorità e valore. Gli sviluppatori portano stime e conoscenza tecnica.

Il team XP e la disponibilità del cliente

- Il team riunisce le competenze necessarie per realizzare e verificare il prodotto.
- Il rappresentante del cliente chiarisce bisogni e priorità con tempi di risposta brevi.
- Il team rende visibile il lavoro e concorda come coordinarsi.
- La co-locazione e il cliente on-site sono pratiche classiche di XP.

In un team remoto occorre progettare esplicitamente accesso alle persone e feedback.

Le dodici pratiche classiche di XP

Area	Pratiche
Pianificazione e cliente	Planning Game, piccoli rilasci, cliente on-site
Design e codice	Metafora del sistema, progettazione semplice, refactoring
Verifica e collaborazione	Testing, pair programming, proprietà collettiva
Continuità del lavoro	Integrazione continua, settimana di 40 ore, standard di codifica

Le formulazioni successive evolvono: “40 ore” diventa attenzione al lavoro sostenibile.

Come le pratiche si sostengono

Pratica	Supporto necessario	Effetto cercato
Refactoring	Test automatici e passi piccoli	Cambiare la struttura con fiducia
Proprietà collettiva	Convenzioni, revisione e integrazione	Condividere conoscenza e responsabilità
Piccoli rilasci	Storie piccole e cliente disponibile	Ottenere feedback sul valore
Pianificazione adattiva	Stime del team e feedback frequente	Rivedere priorità e previsioni

Una pratica isolata può funzionare; l'insieme riduce rischi che le singole pratiche lasciano aperti.

User stories: scheda, conversazione, conferma

- La scheda descrive in breve un bisogno nel linguaggio dell'utente.
- La conversazione chiarisce contesto, vincoli e alternative.
- La conferma rende verificabile il comportamento atteso.
- La documentazione di dettaglio si aggiunge quando serve a capire o verificare il sistema.

"Come…, voglio…, per…" è un formato utile, non un obbligo universale.

Caso guida: un abbonamento al parcheggio

Come studente, voglio acquistare un abbonamento al parcheggio per evitare il pagamento giornaliero.

Criterio di accettazione	**Riscontro verificabile**
Scelta della durata	Un mese costa 30 €, due mesi costano 60 €.
Pagamento approvato	Il sistema conferma l’acquisto e rende disponibile l’abbonamento.
Pagamento rifiutato	Il sistema informa l’utente e non attiva l’abbonamento.

Un backlog comprende lavoro di tipi diversi

Ordine iniziale	Elemento di lavoro	Tipo
1	Acquistare un abbonamento di un mese.	User story
2	Visualizzare gli abbonamenti acquistati.	User story
3	Aggiungere la scelta di due mesi.	User story
4	Correggere una conferma mostrata due volte.	Difetto
5	Automatizzare build e test.	Attività tecnica

L’ordine è un esempio: valore, rischio e dipendenze possono modificarlo.

Stime relative e story points

- Gli story points esprimono una dimensione relativa del lavoro.
- La stima considera sforzo, complessità e incertezza.
- Il team confronta una storia con storie già comprese.
- Una previsione di consegna richiede anche capacità disponibile ed esperienza del team.

Un punto non è un’ora. I punti di team diversi non sono una misura confrontabile.

Il Planning Game

Cliente / business	Sviluppatori
Chiarisce il valore e sceglie le priorità.	Chiariscono fattibilità, stime e rischi.
Valuta alternative di ambito.	Segnalano dipendenze e vincoli tecnici.
Esamina l’incremento e il feedback.	Usano il lavoro svolto per migliorare le previsioni.

Il piano viene rivisto a ogni iterazione. Le scelte combinano valore, costo e rischio.

Un cambiamento durante l’iterazione

Il cliente chiede: “Vorrei anche l’abbonamento annuale.”

- Chiarire il bisogno e la sua urgenza.
- Valutare costo, rischio e dipendenze con il team.
- Concordare che cosa anticipare, rinviare o rinegoziare.
- Aggiornare il piano e rendere visibile la decisione.

Accogliere il cambiamento significa valutarlo, con responsabilità su tempi e qualità.

Test di accettazione: il comportamento per l’utente

Scenario	Dato	Quando	Allora
Pagamento approvato	Studente autenticato; durata un mese	Il pagamento è approvato	Abbonamento attivo e conferma a 30 €
Pagamento rifiutato	Studente autenticato; durata un mese	Il pagamento è rifiutato	Messaggio di errore e nessun abbonamento attivo

Cliente e team chiariscono le regole. Il team le traduce in verifiche ripetibili.

TDD: un ciclo breve di programmazione

Passo	Azione	Controllo
Red	Scrivere un piccolo test del comportamento successivo.	Il test fallisce per la ragione attesa.
Green	Scrivere il codice minimo che fa passare il test.	Passano il nuovo test e quelli esistenti.
Refactor	Migliorare la struttura senza cambiare il comportamento.	I test rimangono verdi.

Si ripete il ciclo in piccoli passi. La user story può richiedere molti cicli.

Primo ciclo: il prezzo di un mese

Red: definire il risultato atteso

```
def test_un_mese():
    assert prezzo(1) == 30
```

Green: soddisfare il primo test

```
def prezzo(mesi):
    return 30
```

Prima del codice, il test fallisce perché prezzo non è definita.
Dopo questa implementazione, il test passa.

Il primo test non dimostra ancora il comportamento per durate diverse.

Secondo ciclo: due mesi e refactoring

```
def test_due_mesi():
    assert prezzo(2) == 60

def prezzo(mesi):
    return 30 * mesi
```

Il nuovo test fallisce con return 30. La moltiplicazione fa passare entrambi i test.

```
TARIFFA_MENSILE = 30

def prezzo(mesi):
    return TARIFFA_MENSILE * mesi
```

Il refactoring nomina la tariffa e preserva il comportamento. I test restano verdi.

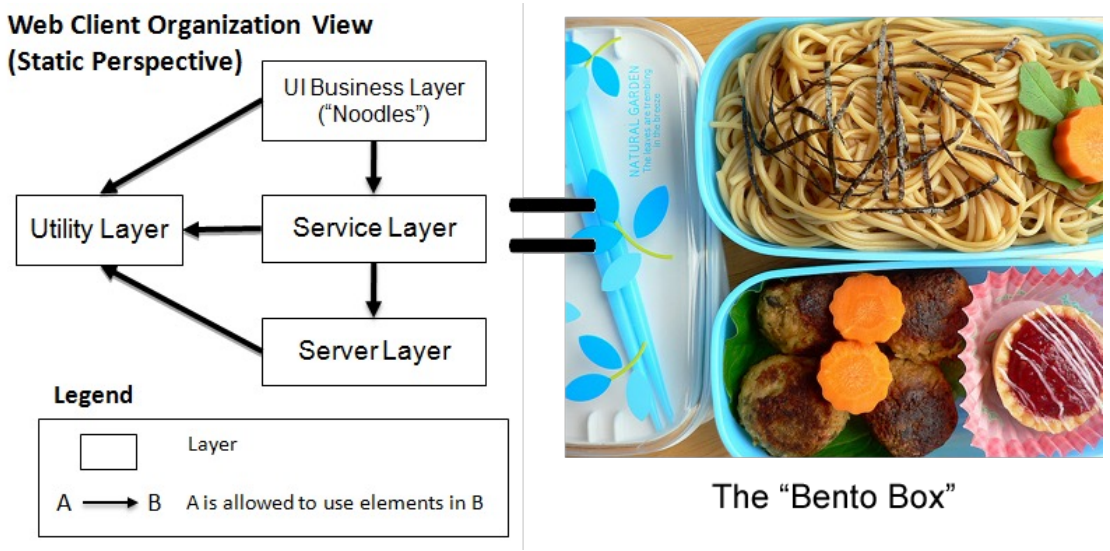
Piccoli incrementi e rilasci frequenti

- Un incremento realizza un comportamento utilizzabile e verificato.
- La prima versione può offrire un mese di abbonamento e un solo metodo di pagamento.
- Le versioni successive aggiungono alternative sulla base del feedback.
- Un’iterazione scandisce il lavoro; un rilascio rende disponibile una versione agli utenti.

Piccolo ambito e qualità adeguata devono convivere.

La metafora del sistema e il linguaggio comune

La metafora classica di XP aiuta il team a condividere una visione.
Un glossario esplicito chiarisce nomi, responsabilità e relazioni.



The "Bento Box"

Progettazione semplice e YAGNI

- Il design deve sostenere i requisiti conosciuti e i vincoli rilevanti.
- Evitare generalizzazioni speculative che aggiungono complessità.
- Valutare presto i rischi tecnici che potrebbero bloccare il prodotto.
- Documentare le decisioni quando aiuta a capirle e mantenerle.

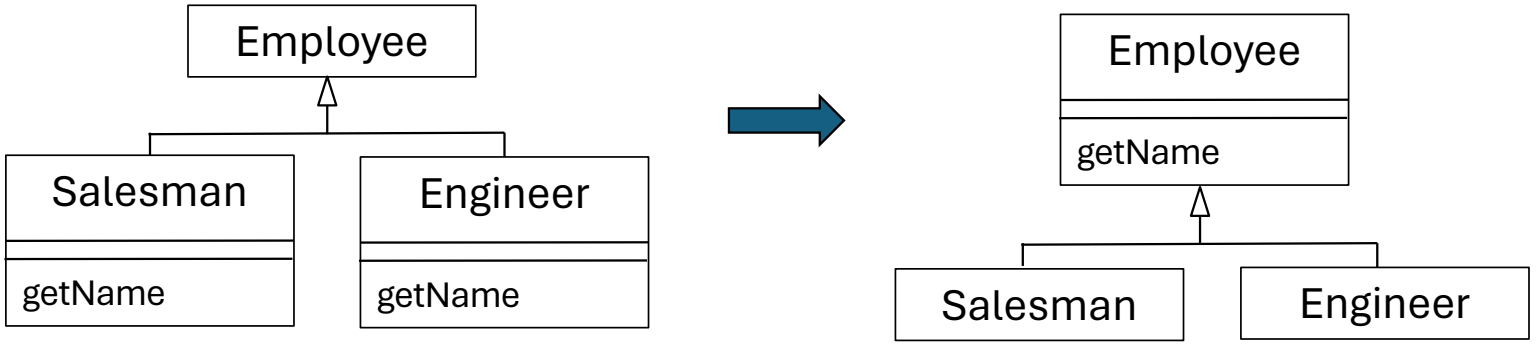
YAGNI: You Aren’t Gonna Need It. La semplicità richiede giudizio tecnico.

Refactoring: cambiare la struttura

- Migliorare la struttura interna preservando il comportamento osservabile.
- Procedere con trasformazioni piccole e verificabili.
- Rendere chiari nomi e responsabilità; eliminare duplicazioni pertinenti.
- Usare test e revisione per ridurre il rischio di regressioni.

Correggere un difetto o aggiungere una funzionalità cambia il comportamento.

Refactoring: un metodo comune



Le sottoclassi duplicano lo stesso comportamento

Il metodo comune viene spostato nella superclasse, preservando il comportamento.
Eliminare le ridondanze nel codice è importante

Pair programming: driver e navigatore

Ruolo	Attività
Driver	Scrive il codice e mantiene il passo operativo.
Navigatore	Ragiona sulle alternative, osserva i rischi e controlla la direzione.
Entrambi	Discutono, verificano le decisioni e scambiano i ruoli.

- Il cambio dei ruoli deve essere abbastanza frequente da coinvolgere entrambi.
- La rotazione delle coppie può diffondere conoscenze nel team.

Pair programming: cosa mostra l’evidenza

Dimensione	Sintesi della meta-analisi di Hannay et al. (2009)
Qualità	Un miglioramento medio piccolo.
Durata del compito	Una riduzione media del tempo di completamento.
Sforzo totale	Un aumento medio del lavoro in ore-persona.
Variabilità	Risultati diversi tra studi; contano complessità ed esperienza.

Benefici e costi dipendono dal contesto. Durata e sforzo sono misure diverse.

Integrazione continua

- Integrare frequentemente modifiche piccole nella linea condivisa del codice.
- Verificare ogni integrazione con build e test automatici.
- Rendere immediatamente visibile un errore e correggerlo rapidamente.
- Limitare la divergenza tra copie e rami di sviluppo.

Una pipeline automatica aiuta. La pratica richiede anche integrazione frequente.

Proprietà collettiva del codice

- Il team condivide conoscenza e responsabilità sul codice.
- Chiunque può proporre e realizzare modifiche, secondo le regole concordate.
- Convenzioni, test e revisione rendono le modifiche comprensibili.
- L’integrazione frequente rende visibili gli effetti sul lavoro degli altri.

La responsabilità condivisa richiede disciplina e tracciabilità.

Un ritmo di lavoro sostenibile

- Il team deve poter mantenere un ritmo ragionevole nel tempo.
- Stanchezza e sovraccarico rendono più difficile ragionare e verificare.
- Le difficoltà devono essere visibili prima di diventare emergenze ricorrenti.
- Priorità e capacità disponibile vanno riconciliate, anche riducendo l'ambito.

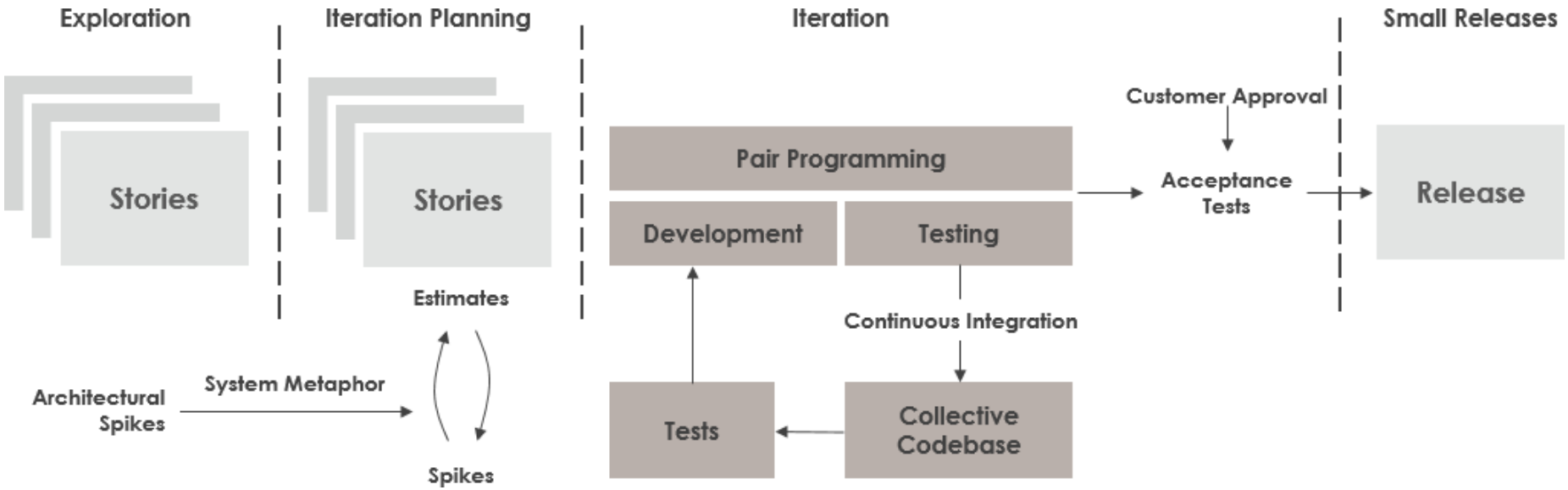
La "settimana di 40 ore" è la formulazione storica di una pratica XP.

Convenzioni di codifica e commenti

- Convenzioni condivise aiutano a leggere e modificare il codice altrui.
- Nomi chiari e struttura semplice rendono visibile l’intenzione.
- I commenti spiegano motivazioni, vincoli e scelte poco evidenti.
- Commenti e documentazione devono evolvere con il codice.

Un commento utile può spiegare perché una soluzione apparentemente ovvia è sbagliata.

XP: dal bisogno all'incremento



Esplorazione, pianificazione, sviluppo e verifica si ripetono con feedback frequente.

IA generativa: supporto alle pratiche XP

Attività	Possibile supporto	Verifica richiesta
Storie	Proporre formulazioni e scomposizioni.	Bisogno e priorità con cliente e team.
Test	Suggerire casi normali e casi limite.	Oracolo, copertura pertinente e risultati.
Codice	Proporre implementazioni e alternative.	Correttezza, sicurezza e comprensibilità.
Refactoring	Proporre nomi, estrazioni e semplificazioni.	Comportamento preservato e design adeguato.

Il risparmio dipende anche dal costo di revisione e dalle correzioni necessarie.

Un errore condiviso tra codice e test generati

Regola concordata: due mesi costano 60 €.
Il modello propone uno sconto che nessuno ha richiesto.

```
# Codice suggerito
def prezzo(mesi):
    return 30 * mesi * 0.9

# Test suggerito
assert prezzo(2) == 54
```

Il test passa, ma conferma la stessa assunzione errata del codice.

Il criterio concordato fornisce un controllo esterno:
prezzo(2) deve essere 60.

Codice e test coerenti tra loro possono essere entrambi sbagliati.

Un’organizzazione del lavoro per il progetto del corso

- Repository condiviso, responsabilità di revisione e permessi espliciti.
- Storie, decisioni e test tracciati negli strumenti del team.
- Un breve coordinamento quotidiano su avanzamento e impedimenti.
- Build e test automatici; dimostrazioni periodiche e riflessione sul lavoro.

Queste sono proposte organizzative da concordare per il corso, non requisiti universali di XP.

Esercizio: dal bisogno ai test

- Scrivere una storia per modificare la durata dell’abbonamento.
- Definire tre criteri di accettazione, incluso un caso non valido.
- Scegliere un incremento piccolo e spiegare la priorità.
- Implementare un ciclo TDD in coppia e mostrare un refactoring.
- Eseguire i test, integrare il codice e descrivere il feedback da raccogliere.

Se usate un LLM, indicate una proposta accettata e una corretta o rifiutata.

Autoverifica: spiegare e applicare

- Quale ipotesi verifica un MVP? Come osserveresti il risultato?
- Quali pratiche XP aiutano a mantenere semplice il codice?
- In cosa differiscono un test unitario e un test di accettazione?
- Perché un test generato insieme al codice può non bastare?
- Che cosa cambieresti nel piano dopo un nuovo requisito urgente?

Riferimenti essenziali

Fondamenti
Manifesto Agile e dodici principi (2001): agilemanifesto.org
Beck & Andres (2004). Extreme Programming Explained, 2ª ed.
Beck (2003). Test Driven Development: By Example.

Pratiche e organizzazione
Schwaber & Sutherland (2020). The Scrum Guide: scrumguides.org
Fowler (2018). Refactoring, 2ª ed.: refactoring.com
Fowler. Continuous Integration: martinfowler.com
Wilson. Building Software Together: buildtogether.tech

Evidenza e approfondimenti
Hannay et al. (2009). The effectiveness of pair programming: A meta-analysis. Information and Software Technology, 51(7), 1110–1122.
Agile Alliance Glossary: agilealliance.org/agile101/agile-glossary/

Domande e discussione

Quale pratica introdurresti per prima
nel tuo team, e quale problema risolverebbe?